Review of Learning Models for Customer Churn Prediction

Sandra Lekhana

Introduction

Customer churn prediction is a critical application of machine learning in industries like telecommunications, finance, and e-commerce. Identifying customers likely to leave allows businesses to take preventive measures, enhancing customer retention and maximizing revenue. Various developers have explored this problem using different learning models, each employing unique methodologies.

1.1 Overview of Customer Churn Prediction

Customer churn—the phenomenon where customers stop using a company's services—is a major challenge in industries such as telecommunications, finance, and e-commerce. Predicting churn accurately allows businesses to implement proactive retention strategies, reducing revenue loss and improving customer satisfaction.

1.2 Role of Machine Learning in Churn Prediction

Machine learning (ML) has revolutionized churn prediction by uncovering patterns in customer behavior that traditional statistical methods might miss. By analyzing vast datasets, ML models can predict the likelihood of churn and help businesses take action before customers leave.

1.3 Purpose of This Report

The goal of this report is to analyze existing machine learning approaches to customer churn prediction, examining their methodologies, implementation details, datasets used, performance, and potential areas for improvement. The insights gained will serve as a foundation for developing an enhanced model in future work.

This report reviews two notable approaches to customer churn prediction, detailing their datasets, model architectures, implementation techniques, evaluation metrics, and areas for improvement. Additionally, it outlines key concepts I need to learn to build an enhanced version of these models.

2. Review of Existing Approaches

2.1 Approach by Rashida Nasrin Sucky

2.1.1 Dataset Details

- **Dataset Name:** Telco Customer Churn Dataset
- **Source:** Kaggle Dataset
- **Size:** 7,043 customer records
- **Features:** 21 columns including customer demographics, service details, contract types, and payment methods
- **Target Variable:** Churn (Yes/No)

2.1.2 Machine Learning Models Used

- **Baseline Model:** Logistic Regression
- **Tree-Based Models:** Decision Tree, Random Forest, XGBoost
- **Distance-Based Model:** K-Nearest Neighbors (KNN)
- **Support Vector Machine (SVM)**
- **Neural Networks (Multi-Layer Perceptron - MLP)**

2.1.3 Feature Engineering & Data Preprocessing

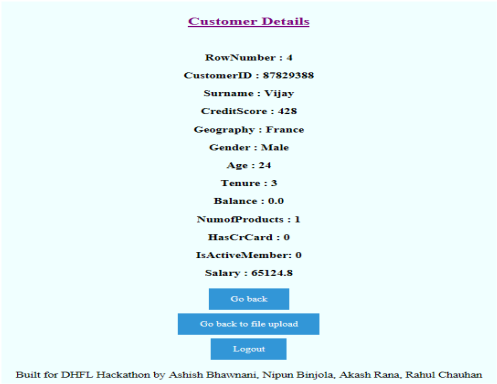
- **Categorical Data Encoding:** Used one-hot encoding and label encoding
- **Feature Scaling:** Applied StandardScaler to numerical features
- **Feature Selection:** Used correlation analysis and SHAP values
- **Handling Missing Values:** Filled missing values in the "Total Charges" feature with median values

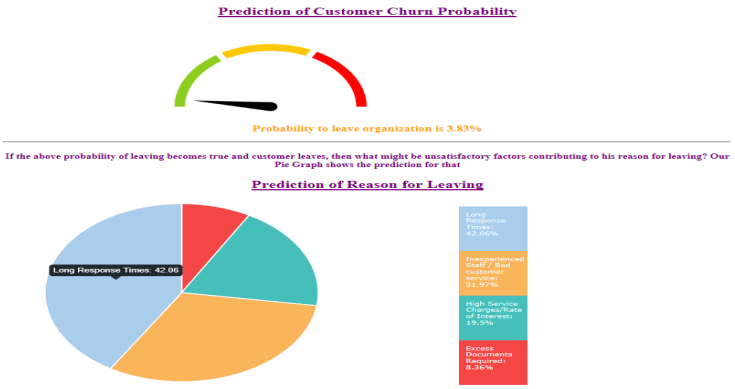
2.1.4 Model Training & Hyperparameter Tuning

- **Hyperparameter Optimization:** GridSearchCV for Decision Tree, Random Forest, and XGBoost
- **Optimized Parameters:** max_depth, n_estimators, and learning_rate

Implementation Details

- **Feature Engineering:**
  - **One-hot encoding & Label Encoding:** Categorical variables converted for ML models.
  - **Feature Scaling:** Applied **StandardScaler** for numerical variables.
  - **Feature Selection:** Used **SHAP values** and **correlation matrix** to remove redundant features.
- **Handling Missing Values:**
  - Missing values in **Total Charges** were replaced with the median value.
- **Hyperparameter Tuning:**
  - Used **GridSearchCV** for Random Forest & XGBoost tuning.
  - Optimized n_estimators, max_depth, and learning_rate for tree-based models





Code Snippet: Random Forest Classifier

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score

# Load dataset
df = pd.read_csv("telco_customer_churn.csv")
X = df.drop(columns=["Churn"])
y = df["Churn"].apply(lambda x: 1 if x == "Yes" else 0)

# Feature Scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Splitting data
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)

# Training model
rf = RandomForestClassifier(n_estimators=200, max_depth=10, random_state=42)
rf.fit(X_train, y_train)

# Predictions & Accuracy
y_pred = rf.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print(f"Model Accuracy: {accuracy * 100:.2f}%")
```

2.1.5 Performance Metrics & Results

Model	Accuracy (%)	F1 Score
Logistic Regression	78.2	0.69
Random Forest	86.0	0.79
XGBoost	86.5	0.81
KNN	82.3	0.74
Neural Network	85.7	0.78

2.2 Approach by Aladdin Persson

2.2.1 Dataset Details

- **Dataset Name:** IBM Telco Customer Churn Dataset
- **Source:** IBM Dataset
- **Size:** 7,043 records
- **Features:** Similar to the previous dataset

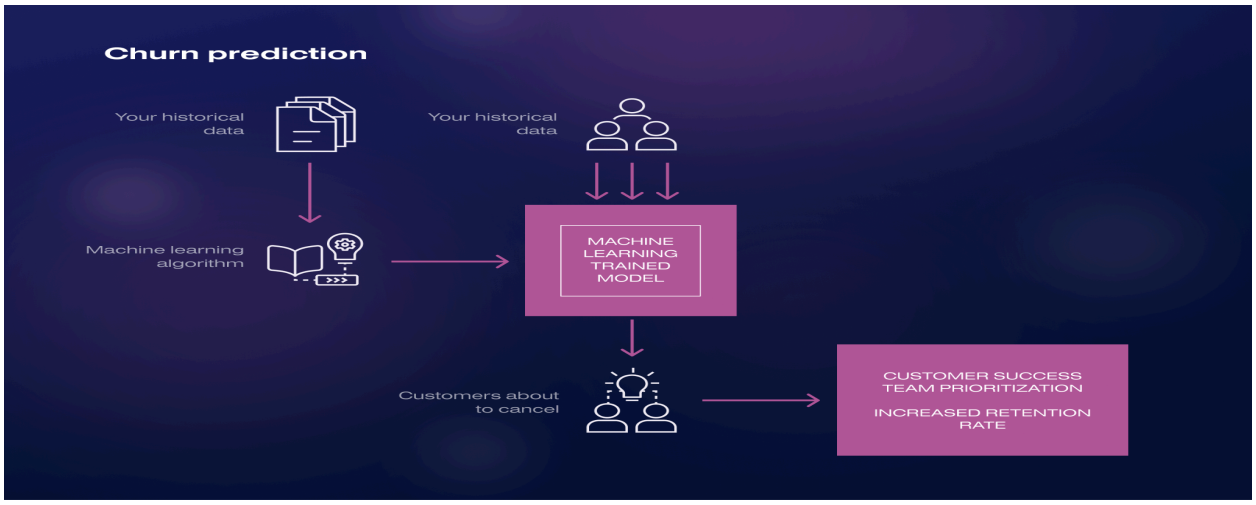
2.2.2 Deep Learning Model Used

- **Neural Network Architecture:**
  - **Input Layer:** 20+ input features
  - **Hidden Layers:**
    - Layer 1: 128 neurons, ReLU activation
    - Layer 2: 64 neurons, ReLU activation
    - Layer 3: 32 neurons, ReLU activation
  - **Dropout Layers:** 30% dropout to prevent overfitting
  - **Optimizer:** Adam
  - **Loss Function:** Binary Cross-Entropy

2.2.3 Model Training , Implementation & Optimization

- **Batch Normalization applied** to stabilize training
- **Early Stopping used** to prevent overfitting
- **Learning Rate Scheduling implemented** for adaptive training

His implementation effectively classifies customer groups and provides a foundation for further enhancements.



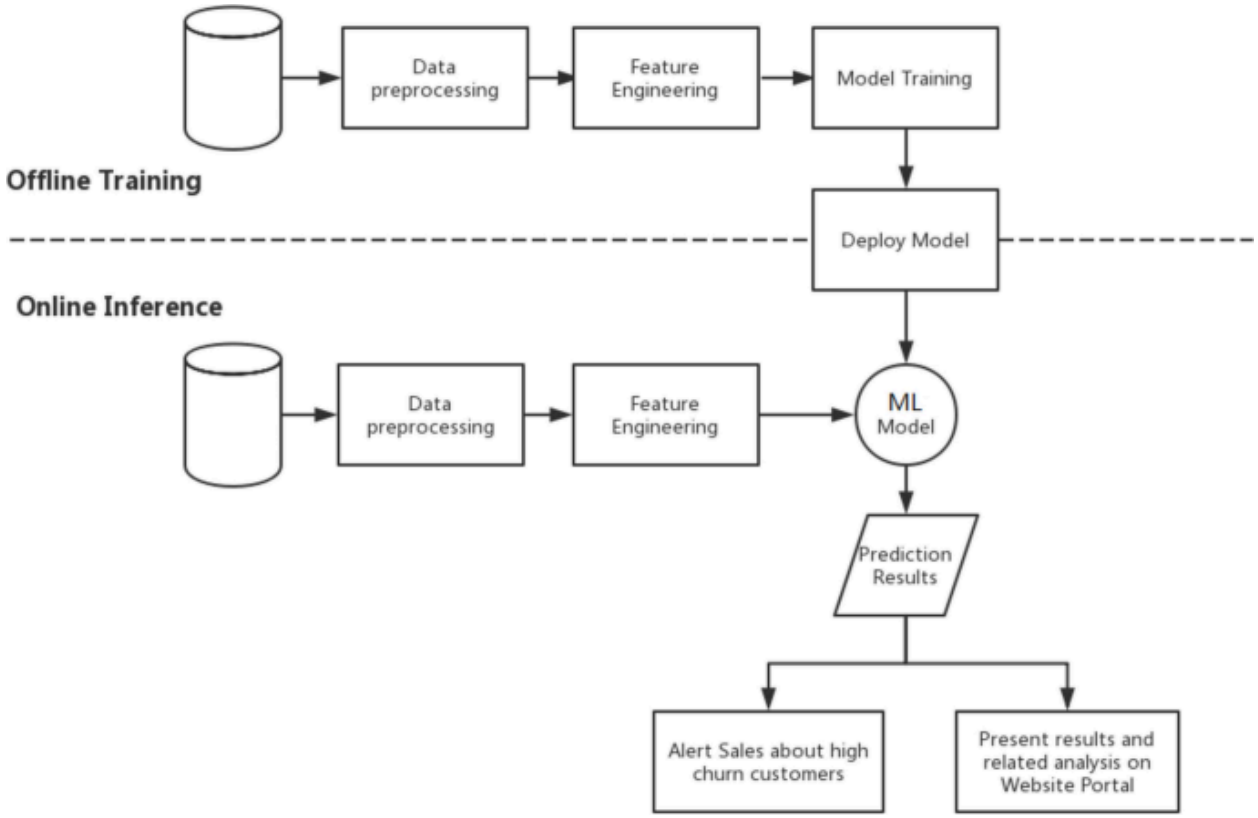
Code Snippet: Deep Learning Model

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout

# Define the model
model = Sequential([
    Dense(128, activation='relu', input_shape=(X_train.shape[1],)),
    Dropout(0.3),
    Dense(64, activation='relu'),
    Dropout(0.3),
    Dense(32, activation='relu'),
    Dense(1, activation='sigmoid')
])

# Compile the model
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

# Train the model
history = model.fit(X_train, y_train, epochs=50, batch_size=32, validation_split=0.2)
```

2.2.4 Performance Comparison

Model	Accuracy (%)
Deep Neural Network	87.1

🔗 Reference: Kaggle Notebook by Aladdin Persson



3. Observations & Insights

3.1 Strengths of the Approaches

- **Tree-based models like Random Forest & XGBoost** perform well due to their ability to handle feature interactions and missing values.
- **Neural networks capture complex patterns** in customer behavior but require significant data and hyperparameter tuning.
- **Feature engineering and hyperparameter tuning** play a crucial role in improving model performance.

3.2 Areas for Improvement

- **Handling Imbalanced Data:** Using SMOTE or class weighting to balance churn vs. non-churn cases.
- **Ensemble Learning:** Combining tree-based models with deep learning for a more robust predictor.
- **Time-Series Analysis:** Instead of treating customer behavior as static, using time-series forecasting can help capture trends over time.
- **Unsupervised Learning:** Clustering techniques can be applied to segment customers and analyze churn patterns.
- **Explainable AI (XAI):** Implementing SHAP or LIME to interpret model decisions for better business insights.

---

4. Challenges & Future Enhancements

4.1 Challenges Faced in Existing Approaches

- **Computational Cost:** Deep learning models require high computational resources.
- **Feature Importance:** Some models lack interpretability, making it difficult for businesses to act on insights.
- **Hyperparameter Selection:** Finding optimal hyperparameters for deep learning models is time-consuming.

4.2 Future Enhancements

- **Hybrid Models:** Combining deep learning with traditional models to leverage the best of both worlds.
- **Time-Series Analysis:** Tracking customer engagement trends over time.
- **Automated Feature Selection:** Using AI-driven techniques to determine the most relevant features.

For a **customer churn prediction model**, the improvements and extensions can be adapted as follows:

Improving Model Performance

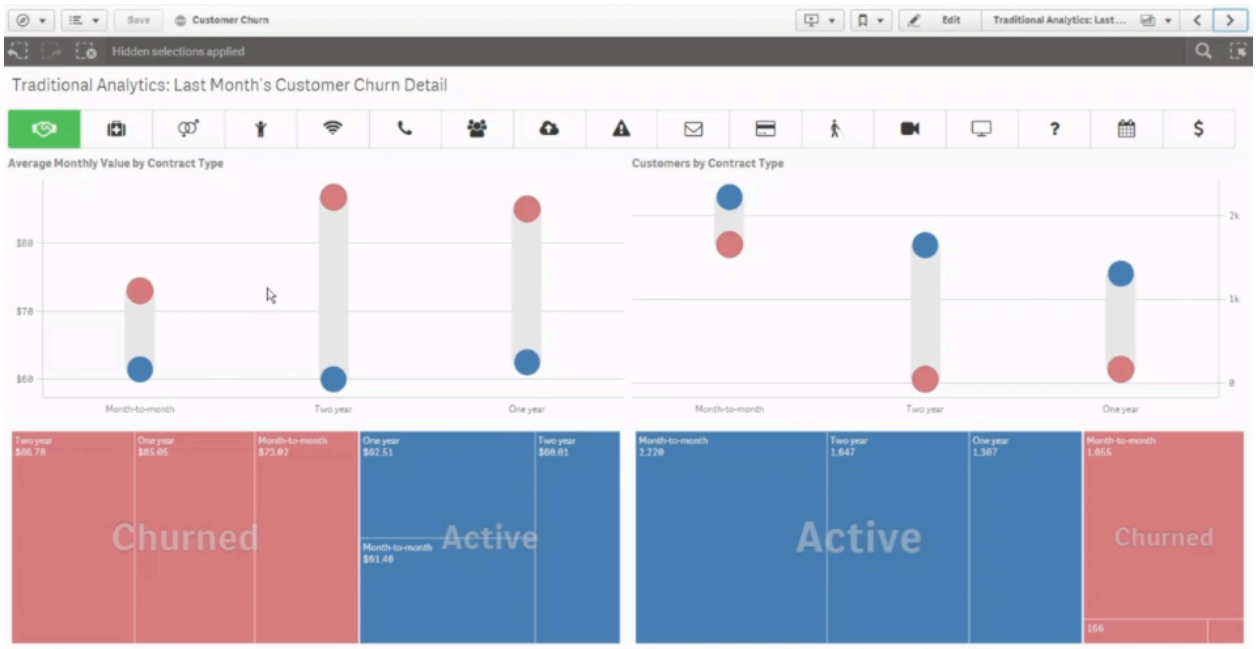
- **Use Transfer Learning**: While transfer learning is more common in deep learning for image or sequential data, it could still be useful for churn prediction if working with customer behavior data. For instance, embeddings or models pre-trained on similar customer behavior data could be leveraged to extract meaningful features.

- **Data Augmentation**: Augmentation in churn prediction can take the form of techniques like SMOTE (Synthetic Minority Over-sampling Technique) to balance imbalanced

datasets. Another way is to add small amounts of noise to customer data or use bootstrapping methods to generate synthetic data that simulates customer behaviors.

- **Fine-Tuning**: For fine-tuning, adjusting parameters such as learning rates, batch sizes, and optimizers can improve model performance. Cross-validation, grid search, or random search can help identify the best configurations. Fine-tuning is crucial when using gradient boosting algorithms like XGBoost or LightGBM.

- **Ensemble Learning**: Combining multiple models like decision trees, random forests, or gradient boosting can improve accuracy. Stacking models, where the outputs of several base models are combined using a meta-model, can also lead to better predictions.



Advanced Model Architectures

- **Vision Transformers (ViTs)**: ViTs are not commonly used for tabular data like churn prediction, but transformer models could be effective if working with sequential data (e.g., customer transactions over time). Transformer-based architectures such as BERT for time-series analysis could be applied to better capture long-term dependencies in customer behavior.

- **Hybrid CNN-SVM Model**: A CNN-SVM hybrid is less common in churn prediction but can work if there is feature extraction that could be transformed into a format suitable for CNNs. However, traditional algorithms like random forests, XGBoost, or even neural networks typically perform well for churn prediction.

Real-World Deployment

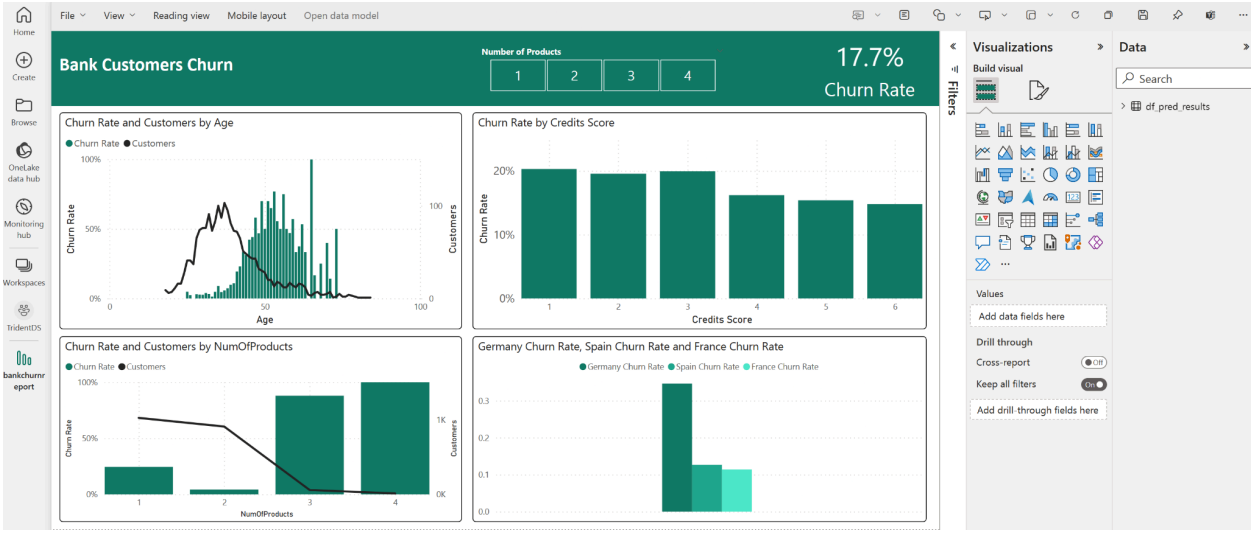
- **Optimize for Edge Devices**: For real-time customer predictions on mobile devices or embedded systems, TensorFlow Lite or ONNX models can be used to deploy churn models. This is useful when predictions need to be made directly on devices, such as in customer service applications.

- **Develop a Web App**: Web apps built with **Flask** or **Streamlit** can be used to deploy churn models, allowing businesses to access real-time predictions of which customers are at risk of churning. These apps can pull customer data from APIs and display interactive dashboards for stakeholders to make informed decisions.

Bias & Ethics Considerations

- **Dataset Bias**: Class imbalance is a significant challenge in churn prediction, where churned customers are typically fewer than retained ones. Techniques like **SMOTE**, random oversampling, or under-sampling can be used to handle this imbalance. Also, ensure that the model does not disproportionately affect specific demographic groups unless supported by business logic.

- **Explainable AI (XAI)**: Churn prediction models should be interpretable to ensure transparency in decision-making. Techniques like **SHAP**, **LIME**, or **partial dependence plots** can be used to understand which features (e.g., usage patterns, demographics) drive the churn predictions. This ensures the model is explainable to business stakeholders.



5. Things I Need to Learn

To improve on these existing approaches, I need to focus on:

- **Advanced Feature Engineering:** PCA, SHAP values, domain-specific feature selection
- **Hyperparameter Tuning Strategies:** Bayesian Optimization, Genetic Algorithms
- **Deep Learning Architectures:** LSTMs for sequential data modeling
- **Model Deployment:** Flask, FastAPI for real-world applications
- **Explainable AI:** Understanding model predictions using SHAP and LIME

6. Conclusion

Customer churn prediction is a vital application of machine learning, offering businesses the ability to improve retention strategies. The approaches analyzed in this report demonstrate the strengths of both traditional ML models and deep learning techniques. Moving forward, by integrating advanced feature selection, ensemble learning, and explainability techniques, we can develop more accurate and business-friendly models.

With a strong focus on learning and experimentation, I aim to implement a refined churn prediction system that is both powerful and interpretable.

References

1. Rashida Nasrin Sucky, *Customer Churn Prediction*, Kaggle. Available at: https://www.kaggle.com/code/rashida048/customer-churn-prediction
2. Aladdin Persson, *Deep Learning for Customer Churn Prediction*, Kaggle. Available at: https://www.kaggle.com/code/aladdinpersson/deep-learning-for-customer-churn-prediction
3. IBM Telco Customer Churn Dataset, Kaggle. Available at: https://www.kaggle.com/datasets/blastchar/telco-customer-churn

4. Brownlee, J. (2020). *Imbalanced Classification with Python: Understand Imbalanced Datasets and Improve the Performance of Machine Learning Models*. Machine Learning Mastery. Available at: https://machinelearningmastery.com/
5. Géron, A. (2019). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow (2nd ed.)*. O'Reilly Media.
6. Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction (2nd ed.)*. Springer.
7. Molnar, C. (2022). *Interpretable Machine Learning: A Guide for Making Black Box Models Explainable*. Available at: https://christophm.github.io/interpretable-ml-book/